

Examen final de Programació i Estructures de Dades (PRD)

Data: 12 de juny de 2024. Duració: 2h 30min.

Publicació de notes provisionals: 19/06/2024 (Atenea)

Revisió de l'examen: 20/06/2024 – 11:00h, Sala D6-221 (edifici D6, 2ª planta)

IMPORTANT: cal resoldre i lliurar els problemes 1, 2 i 3 en fulles separades

PROBLEMA 1: 4 punts, temps de resolució: 50 min.

Volem implementar un programa que gestioni les habitacions d'un hotel, permetent reservar diverses habitacions juntes (números d'habitació consecutius) en una mateixa planta. El disseny del programa contempla la definició de les constants simbòliques i estructures de dades següents:

```
#define NUM_HAB 20      /* Número màxim d'habitacions per planta */
#define NUM_PLAN 5      /* Número de plantes de l'hotel */

#define LLIURE 0
#define OCUPADA 1

#define OK 0
#define ERROR -1

#define SINGLE 1
#define DOUBLE 2

typedef struct {
    int max_ocup; /* SINGLE o DOUBLE */
    int estat; /* LLIURE o OCUPADA */
    int ocupants; /* 0, 1 o 2 ocupants */
}t_habitacio;

typedef struct {
    t_habitacio habs[NUM_PLAN][NUM_HAB]; /* Matriu d'habitacions */
    int num_habs[NUM_PLAN]; /* Núm. d'habitacions a cada planta */
}t_hotel;

void inicialitzar_hotel (t_hotel *hotel);
int habs_juntes (t_hotel hotel, int num_habs, int *planta, int *hab);
int reservar_habitacio (t_habitacio *hab, int num_ocup);
int reserva_multiple (t_hotel *hotel, int num_habs, int planta, int hab_ini,
                     int ocup[]);
void mostrar_hotel (t_hotel hotel);
```

L'hotel té NUM_PLAN plantes i pot arribar a tenir un màxim de NUM_HAB habitacions per planta. El nombre d'habitacions de cada planta de l'hotel es guarda a les posicions del vector num_habs, camp de l'struct t_hotel. Podeu assumir que les plantes de l'hotel i les habitacions de cada planta s'enumeren consecutivament, començant des de 0.

A partir d'aquesta descripció inicial, cal implementar les funcions següents:

- a. **(0.5 punts)** void inicialitzar_hotel (t_hotel *hotel); aquesta funció ha de inicialitzar l'hotel de la manera següent: inicialment, totes les habitacions estan lliures i tenen un nombre d'ocupants igual a 0. L'ocupació màxima d'una habitació es calcula multiplicant el número de planta pel número d'habitació. Al número

resultant cal sumar-li 2 i calcular el mòdul 2 del número final. Si el valor resultant és 0, l'habitació és de tipus SINGLE. Altrament, l'habitació és de tipus DOUBLE. Pel que fa a les plantes de l'hotel, aquestes tenen 10, 15, 20, 8 i 5 habitacions, respectivament.

```
void inicializar_hotel (thotel *hotel)
{
    int i, j;

    hotel->num_habs[0] = NUM_HAB - 10;
    hotel->num_habs[1] = NUM_HAB - 5;
    hotel->num_habs[2] = NUM_HAB;
    hotel->num_habs[3] = NUM_HAB - 12;
    hotel->num_habs[4] = NUM_HAB - 15;

    for (i = 0; i < NUM_PLAN; i++)
    {
        for (j = 0; j < hotel->num_habs[i]; j++)
        {
            hotel->habs[i][j].estat = LLIURE;
            hotel->habs[i][j].ocupants = 0;
            if ((i*j + 2)%2 == 0)
                hotel->habs[i][j].max_ocup = SINGLE;
            else
                hotel->habs[i][j].max_ocup = DOUBLE;
        }
    }
}
```

- b. **(1.5 punts)** int habs_juntes (t_hotel hotel, int num_habs, int *planta, int *hab); aquesta funció ha de comprovar si a l'hotel hi ha num_habs habitacions lliures consecutives en una mateixa planta. En cas afirmatiu, la funció retorna OK, informant de la planta i número inicial d'habitació lliure a través dels paràmetres planta i hab, respectivament. Si el valor del paràmetre num_habs és incorrecte, o no hi ha prou habitacions lliures consecutives en cap de les plantes de l'hotel, la funció retorna ERROR.

```
int habs_juntes (t_hotel hotel, int num_habs, int *planta, int *hab)
{
    int i, j, k, cont = 0;

    *planta = 0;
    *hab = 0;

    if (num_habs < 0 || num_habs > NUM_HAB)
        return (ERROR);

    for (i = 0; i < NUM_PLAN; i++)
    {
        for (j = 0; j < hotel.num_habs[i]; j++)
        {
            if (j + num_habs <= hotel.num_habs[i])
            {
                for (k = 0; k < num_habs; k++)
                {
                    if (hotel.habs[i][j+k].estat == LLIURE)
                        cont++;
                }
            }
        }
    }
}
```

```

    }
    if (cont == num_habs)
    {
        *planta = i;
        *hab = j;
        return (OK);
    } else
        cont = 0;
}
}
return (ERROR);
}

```

- c. **(0.5 punts)** `int reservar_habitacio (t_habitacio *hab, int num_ocup);` aquesta funció ha d'intentar reservar l'habitació apuntada pel paràmetre `hab` amb un nombre d'ocupants igual al paràmetre `num_ocup`. Si l'habitació ja estava ocupada prèviament, o el valor del paràmetre `num_ocup` és incorrecte (≤ 0 o excedeix la capacitat màxima de l'habitació), la funció cal que retorni `ERROR`. En cas contrari, cal que marqui l'habitació com a ocupada i en guardi el seu nombre d'ocupants, per tal d'acabar retornant `OK`.

```

int reservar_habitacio (t_habitacio *hab, int num_ocup)
{
    if (hab->estat == OCUPADA)
        return (ERROR);

    if (num_ocup <= 0 || hab->max_ocup < num_ocup)
        return (ERROR);

    hab->estat = OCUPADA;
    hab->ocupants = num_ocup;
    return (OK);
}

```

- d. **(1 punt)** `int reserva_multiple (t_hotel *hotel, int num_habs, int planta, int hab_ini, int ocup[]);` aquesta funció cal que reservi un nombre d'habitacions lliures consecutives igual al paràmetre `num_habs`, a la planta indicada pel paràmetre `planta` i des de l'habitació inicial indicada pel paràmetre `hab_ini`. L'ocupació específica de cada habitació s'indica a les posicions respectives del vector `ocup`, també passat com a paràmetre. Cal comprovar que els valors dels paràmetres `num_habs`, `planta` i `hab_ini` són correctes. A més, cal comprovar que l'ocupació pretesa de cada habitació no excedeix la seva ocupació màxima. En cas de detectar qualsevol error en aquestes comprovacions, cal que la funció retorni `ERROR`. Altrament, cal marcar les habitacions com a ocupades i guardar l'ocupació de cadascuna d'elles, d'acord amb els valors de les posicions del vector `ocup`. Finalment, la funció ha de retornar `OK`.

```

int reserva_multiple (t_hotel *hotel, int num_habs, int planta, int hab_ini,
                    int ocup[])
{
    int i, j;

    if (num_habs < 0 || num_habs > NUM_HAB)

```

```

        return (ERROR);

    if (planta < 0 || planta >= NUM_PLAN)
        return (ERROR);

    if (hab_ini < 0 || hab_ini >= NUM_HAB)
        return (ERROR);

    if (hab_ini + num_habs > hotel->num_habs[planta])
        return (ERROR);

    for (i = hab_ini; i < num_habs + hab_ini; i++)
        if (hotel->habs[planta][i].estat == OCUPADA
            || ocup[i - hab_ini] < 0
            || hotel->habs[planta][i].max_ocup < ocup[i - hab_ini])
            return (ERROR);

    for (i = hab_ini; i < num_habs + hab_ini; i++)
    {
        if (reservar_habitacio(&hotel->habs[planta][i], ocup[i - hab_ini])
            == ERROR)
            return (ERROR);
    }

    return (OK);
}

```

- e. **(0.5 punts)** void mostrar_hotel(t_hotel hotel); aquesta funció ha de mostrar l'ocupació de l'hotel, tal i com s'indica a l'exemple d'execució següent:

```

Planta 0
Habitació 0: SINGLE OCUPADA
Habitació 1: SINGLE OCUPADA
Habitació 2: SINGLE OCUPADA
Habitació 3: SINGLE OCUPADA
Habitació 4: SINGLE OCUPADA
...
Planta 1
Habitació 0: LLIURE de tipus SINGLE
Habitació 1: DOUBLE OCUPADA per 2
Habitació 2: LLIURE de tipus SINGLE
Habitació 3: LLIURE de tipus DOUBLE
Habitació 4: LLIURE de tipus SINGLE
...

```

```

void mostrar_hotel (t_hotel hotel)
{
    int i, j;

    for (i = 0; i < NUM_PLAN; i++)
    {
        printf("Planta %d\n", i);
        for (j = 0; j < hotel.num_habs[i]; j++)
        {
            printf ("Habitació %d: ", j);
            if (hotel.habs[i][j].estat == LLIURE)
            {

```

```

        printf ("LLIURE de tipus ");
        if (hotel.habs[i][j].max_ocup == SINGLE)
            printf("SINGLE\n");
        else
            printf("DOUBLE\n");
    }
    else
    {
        if (hotel.habs[i][j].max_ocup == SINGLE)
            printf("SINGLE OCUPADA\n");
        else
            printf("DOUBLE OCUPADA per %d\n",
                    hotel.habs[i][j].ocupants);
    }
}
}
}

```

PROBLEMA 2: 4.5 punts, temps de resolució: 75 min.

A continuació es detallen les estructures de dades necessàries per a la implementació d'una taula hash d'adreçament tancat que actuï com a registre de pàgines web, identificades per la seva URL (direcció) com a clau. De cada pàgina web, la taula hash n'ha de guardar el seu nombre total de visites rebudes:

```

#define MAX_L    250
#define NUM_C    16

typedef struct
{
    char url[MAX_L];    /* URL (direcció) de la pàgina web */
    int num_vis;        /* Nombre total de visites rebudes */
}t_web;

typedef struct node
{
    t_web web;          /* Pàgina web guardada */
    struct node *next;  /* Punter al següent node */
}t_node;

typedef struct
{
    t_node * caselles[NUM_C]; /* Array intern de caselles */
}t_taula;

```

A partir de les estructures de dades definides, implementa les següents funcions:

- a. **(0.3 punts)** int hashCode(char url[MAX_L]); aquesta funció cal que retorni un valor enter positiu com a codi hash associat a la URL passada com a paràmetre. Per tal d'aconseguir-ho, considera els codis ASCII dels caràcters que componen la URL. Tingues en compte que dues URLs amb els mateixos caràcters però en posicions diferents haurien de donar lloc a codis hash diferents, a ser possible.

```

int hashCode(char url[MAX_L])
{
    int sum = 0, i;

    for (i = 0; i < strlen(url); i++)
        sum += (i+1)*url[i];

    return (sum);
}

```

- b. **(0.7 punts)** `t_web * get(t_taula *taula, char url[MAX_L]);` aquesta funció cal que retorni un punter a la pàgina web amb la URL passada com a paràmetre, en cas que aquesta existeixi a la taula hash apuntada pel paràmetre `taula`. Si no hi existeix, cal que la funció retorni `NULL`.

```

t_web * get(t_taula *taula, char url[MAX_L])
{
    t_node *tmp;
    int c = hashCode(url) % NUM_C;

    tmp = taula->caselles[c];
    while (tmp != NULL)
    {
        if(strcmp(tmp->web.url, url) == 0)
            return (&tmp->web);

        tmp = tmp->next;
    }

    return (NULL);
}

```

- c. **(1 punt)** `int put(t_taula *taula, t_web web);` aquesta funció cal que afegixi la pàgina web passada com a paràmetre a la taula hash apuntada pel paràmetre `taula`. La pàgina web cal que quedi identificada per la seva URL com a clau. Si ja existia una pàgina web amb URL idèntica a la taula hash, cal que la funció retorni directament `-2`. En cas de succeir qualsevol problema durant la reserva de la memòria dinàmica necessària per l'operació, cal que la funció retorni `-1`. Si la operació es realitza correctament, cal que retorni `0`.

```

int put(t_taula *taula, t_web web)
{
    t_node *nou;
    int c = hashCode(web.url) % NUM_C;

    if (get(taula, web.url) != NULL)
        return (-2);

    nou = (t_node *)malloc(sizeof(t_node));
    if (nou == NULL)
        return (-1);
}

```

```

nou->web = web;
nou->next = taula->caselles[c];
taula->caselles[c] = nou;
return (0);

/* També s'accepta afegir el nou node com a darrer de la llista
   enllaçada de la casella c */
}

```

- d. **(1 punt)** int eliminar(t_taula *taula, char url[MAX_L]); aquesta funció cal que elimini la pàgina web amb la URL passada com a paràmetre de la taula hash apuntada pel paràmetre taula. Si la pàgina web no s'ha pogut eliminar de la taula, perquè no existia en aquesta, cal que la funció retorni 0. En canvi, si la pàgina web s'ha pogut eliminar correctament, cal que retorni 1.

```

int eliminar(t_taula *taula, char url[MAX_L])
{
    t_node *tmp, *tmp2;
    int c = hashcode(url) % NUM_C;

    if (get(taula, url) == NULL)
        return (0);

    /* La pàgina web segur que existeix a la taula */
    tmp = taula->caselles[c];
    if (strcmp(url, taula->caselles[c]->web.url) == 0)
    {
        taula->caselles[c] = taula->caselles[c]->next;
        free (tmp);
    }
    else
    {
        while(strcmp(tmp->next->web.url, url) != 0)
            tmp = tmp->next;

        tmp2 = tmp->next->next;
        free (tmp->next);
        tmp->next = tmp2;
    }

    return (1);
}

```

- e. **(1.5 punts)** int get_llista_valors_ordenats (t_taula *taula, t_node **head); aquesta funció cal que retorni una nova llista enllaçada que contingui totes les pàgines web guardades a la taula hash apuntada pel paràmetre taula. En aquesta nova llista enllaçada, les pàgines web han de quedar ordenades en funció del nombre total de visites rebudes, de forma decreixent (és a dir, les pàgines web amb més visites rebudes han de quedar emmagatzemades a les primeres posicions de la llista). La llista enllaçada s'ha de retornar a través del paràmetre head. Si succeeix un problema durant la reserva de la memòria dinàmica necessària per l'operació, cal que la funció retorni -1. Si tot funciona correctament, cal que retorni 0.

Important: En cas de succeir qualsevol error durant la reserva de memòria dinàmica, la funció ha d'assegurar-se d'alliberar tots els blocs de memòria dinàmica prèviament reservats, abans d'acabar retornant -1.

```
int get_llista_valors_ordenats (t_taula *taula, t_node **head)
{
    int i, error = 0;
    t_node *nou, *tmp, *tmp2;
    *head = NULL;

    for (i = 0; i < NUM_C && !error; i++)
    {
        tmp = taula->caselles[i];
        while (tmp != NULL && !error)
        {
            nou = (t_node *)malloc(sizeof(t_node));
            if (nou == NULL)
                error = 1;
            else
            {
                nou->web = tmp->web;
                if (*head == NULL ||
                    (*head)->web.num_vis < nou->web.num_vis)
                {
                    nou->next = *head;
                    *head = nou;
                }
                else
                {
                    tmp2 = *head;
                    while (tmp2->next != NULL &&
                        tmp2->next->web.num_vis > nou->web.num_vis)
                    {
                        tmp2 = tmp2->next;
                    }

                    nou->next = tmp2->next;
                    tmp2->next = nou;
                }
            }
            tmp = tmp->next;
        }
    }

    if (error)
    {
        while (*head != NULL)
        {
            tmp = (*head)->next;
            free (*head);
            *head = tmp;
        }

        return (-1);
    }
    return (0);
}
```


PROBLEMA 3: 1.5 punts, temps de resolució: 25 min.

Implementa la funció:

```
int apagar_bits (unsigned int *seq, unsigned int b1, unsigned int b2);
```

Aquesta funció rep com a paràmetre un punter (seq) a variable entera sense signe (unsigned int), que guarda una seqüència de 32 bits. També rep dues posicions de bit (b1 i b2), el valor de les quals hauria d'estar comprès entre [0, 31] i $b1 \geq b2$, doncs b1 ha de representar una posició igual o més significativa que b2 de la seqüència binària.

En cas que el valor del paràmetre b1 i/o b2 sigui incorrecte, la funció ha d'acabar retornant directament -1. Altrament, ha d'apagar (posar a 0) tots els bits a 1 entre les posicions b1 i b2 (ambdues incloses) de la seqüència de 32 bits apuntada pel paràmetre seq. A més, ha d'acabar retornant el nombre total de bits apagats durant el procés.

Per exemple, si unsigned int seq = 0xFFFFFFFF9; en cridar apagar_bits(&seq, 5, 2); la funció hauria de retornar 3 i seq acabar prenent un valor igual a 0xFFFFF9C1.

Nota: cal aconseguir la funcionalitat desitjada utilitzant exclusivament operadors a nivell de bit. No s'acceptaran respostes que utilitzin operadors aritmètics per tal d'aconseguir el mateix propòsit.

```
int apagar_bits (unsigned int *seq, int b1, int b2)
{
    int i, bits_apagats = 0;
    unsigned int mask;

    if (b1 < 0 || b2 < 0 || b1 > 31 || b2 > 31 || b1 < b2)
        return (-1);

    for (i = b1; i >= b2; i--)
    {
        mask = 1 << i;
        if ((*seq & mask) != 0)
        {
            bits_apagats++;
            *seq = *seq ^ mask;
        }
    }

    return (bits_apagats);
}
```